Тот же прогон.
Меньше памяти.

Qwen2.5-1.5B · NF4 · LoRA · 64 шага
Сохраняем данные, батчи, случайные маски и выходы Soup.

1,343×

ускорение всего soup train

6,657×

снижение пика памяти

10 / 10

проверок пройдено

Почему база занимает 14 345 МиБ

MLP в fp32

Gate, up и промежуточные активации сохраняются до backward.

Полные логиты

Словарь 151 936: логиты, loss и метрики создают большие буферы.

Копии эмбеддинга

Тот же замороженный вес хранится в fp32 и копируется в fp16.

Паддинг

Около 44% токен-слотов на публичных строках — дополнение.

Сохраняем математику. Убираем лишнее.

01

Реальные токены

Убираем паддинг внутри батча.
Порядок dropout RNG прежний.

02

Пересчёт attention

Для backward храним Q/K/V.
Повторяем дешёвый SDPA forward.

03

LoRA + MLP

Входы восстанавливаем из norm.
Маски dropout храним битами.

04

Голова чанками

Loss, энтропия, accuracy и rewind
без полной матрицы логитов.

Девять собственных Triton-кернелов

NF4 · 1 кернел	Распаковывает веса в fp16. Шкала и округление — как в bitsandbytes.
RMSNorm · 2 кернела	Forward и backward: меньше отдельных операций и сохранённых данных.
RoPE · 2 кернела	LoRA-add, вращение и layout Q/K за один запуск в каждую сторону.
LoRA inputs · 3 кернела	Маски упакованы в биты; входы и градиенты восстанавливаются из norm.
Head stats · 1 кернел	CE, энтропия, argmax, лоссы токенов и градиент логитов в одном чанке.

SwiGLU из стартового примера используется с указанием источника; он не засчитывается как наш.

Измерено в одной сессии. По два прогона.

Время soup train

База

88,35 с

Решение

65,80 с

Пик GPU-памяти

База

14 345 МиБ

Решение

2 155 МиБ

Eval loss: 2,04966 → 2,04968 · +0,001% · Компиляция включена во время.

Корректность проверяем отдельно

20

GPU-тестов выходов и градиентов

79

регрессионных тестов Soup

10 / 10

правил допуска пройдено

Собственные кернелы: 13,3% времени обучения при пороге 5%.



Границы и следующий шаг

Точность	Публичные строки и seed проверены. Скрытые строки оценивает жюри.
Компромисс	За пересчёт платим дополнительными GEMM. Это обмен скорости на меньший пик памяти.
Воспроизводимость	Фиксируем SHA, повторяем базу и решение, сохраняем result.json и checks.json.
Следующий шаг	Меньше CPU-запусков и вызовов SDPA по строкам. Подтвердить выигрыш полным замером.